

Final Project Report: Simulated Malware Detection Using YARA Rules

Prepared By: Harsh

Project Title

Simulated Malware in Python with YARA Rule Detection

Objective

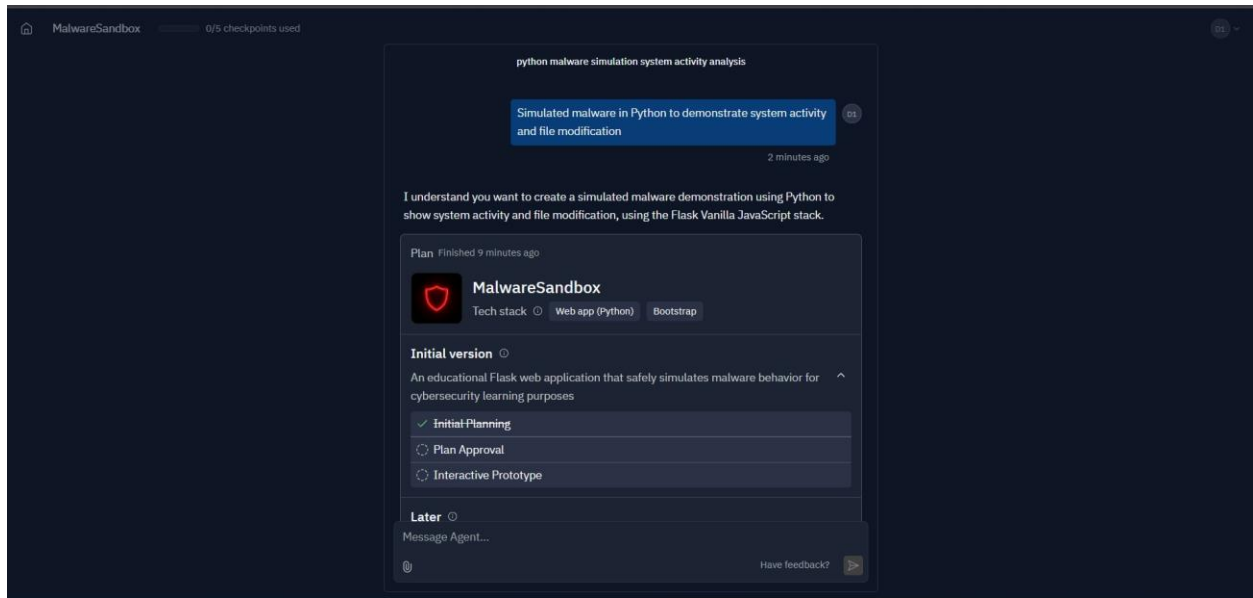
To demonstrate basic malware-like behaviors in a safe, educational sandbox and detect them using a custom YARA rule. The simulation is developed and tested entirely online using Replit.

Tools Used

- **Replit** (for code development and sandboxing)
 - **Python** (Flask web framework)
 - **YARA** (pattern matching for malware detection)
-

Step-by-Step Implementation

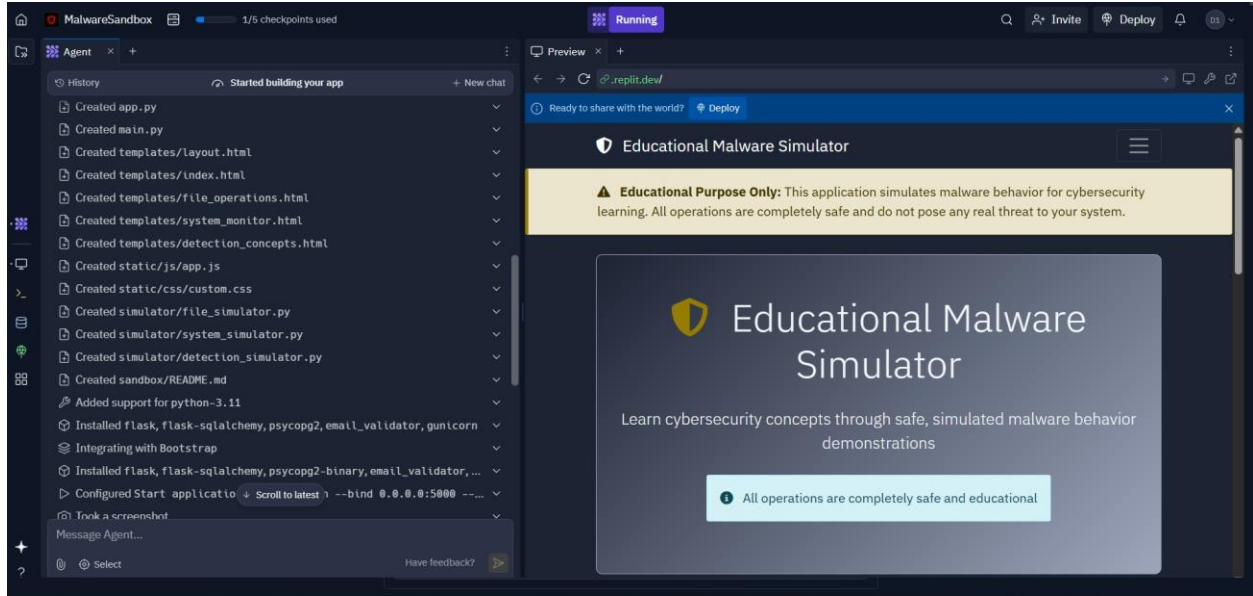
Step 1: Planning and Setup



Created a project on Replit named “MalwareSandbox.”

- Utilized the Replit AI Agent to plan a secure, educational malware simulation.

Step 2: Code Generation and File Structure

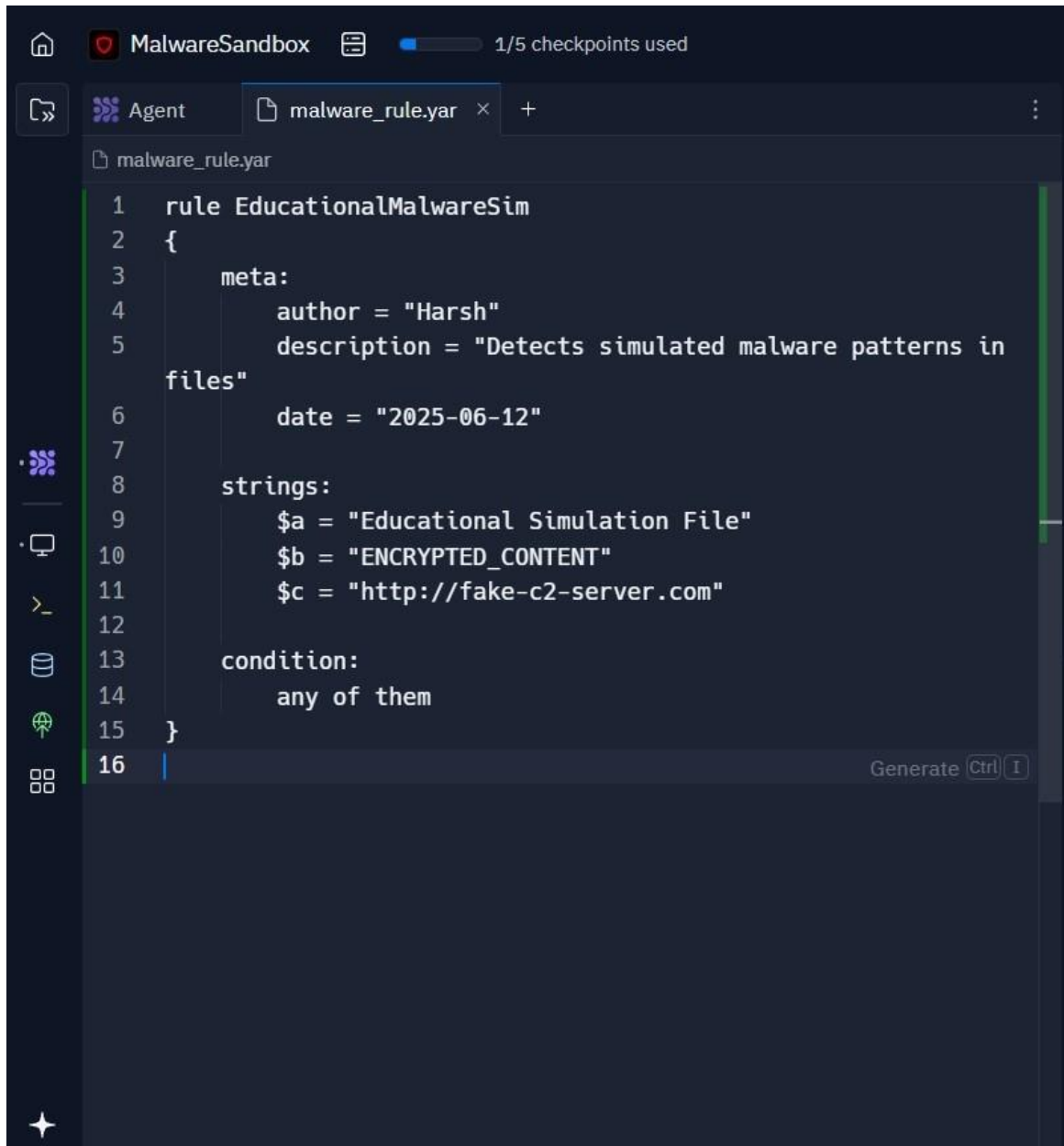


Flask web app structure generated automatically.

- Key components:

- `file_simulator.py`: Handles file creation, deletion, hiding, and encryption.
- `app.py`: Manages Flask routes and application logic.
- HTML/CSS/JS templates: Provides a graphical user interface.

Step 3: User Interface and Web App Preview



The screenshot displays the MalwareSandbox web application interface. At the top, there is a header bar with a home icon, the text "MalwareSandbox", a settings icon, and a progress indicator showing "1/5 checkpoints used". Below the header, there is a tabbed interface with a tab labeled "Agent" and a file named "malware_rule.yar". The main area is a code editor showing the following YARA rule:

```
1 rule EducationalMalwareSim
2 {
3     meta:
4         author = "Harsh"
5         description = "Detects simulated malware patterns in
6         files"
7         date = "2025-06-12"
8     strings:
9         $a = "Educational Simulation File"
10        $b = "ENCRYPTED_CONTENT"
11        $c = "http://fake-c2-server.com"
12
13    condition:
14        any of them
15 }
16
```

On the left side of the code editor, there is a vertical toolbar with icons for file explorer, terminal, command prompt, database, and network. At the bottom right of the code editor, there is a "Generate" button with a keyboard shortcut "Ctrl I".

- Application launched in Replit's preview window.
- Titled **Educational Malware Simulator**, clearly marked for educational use only.

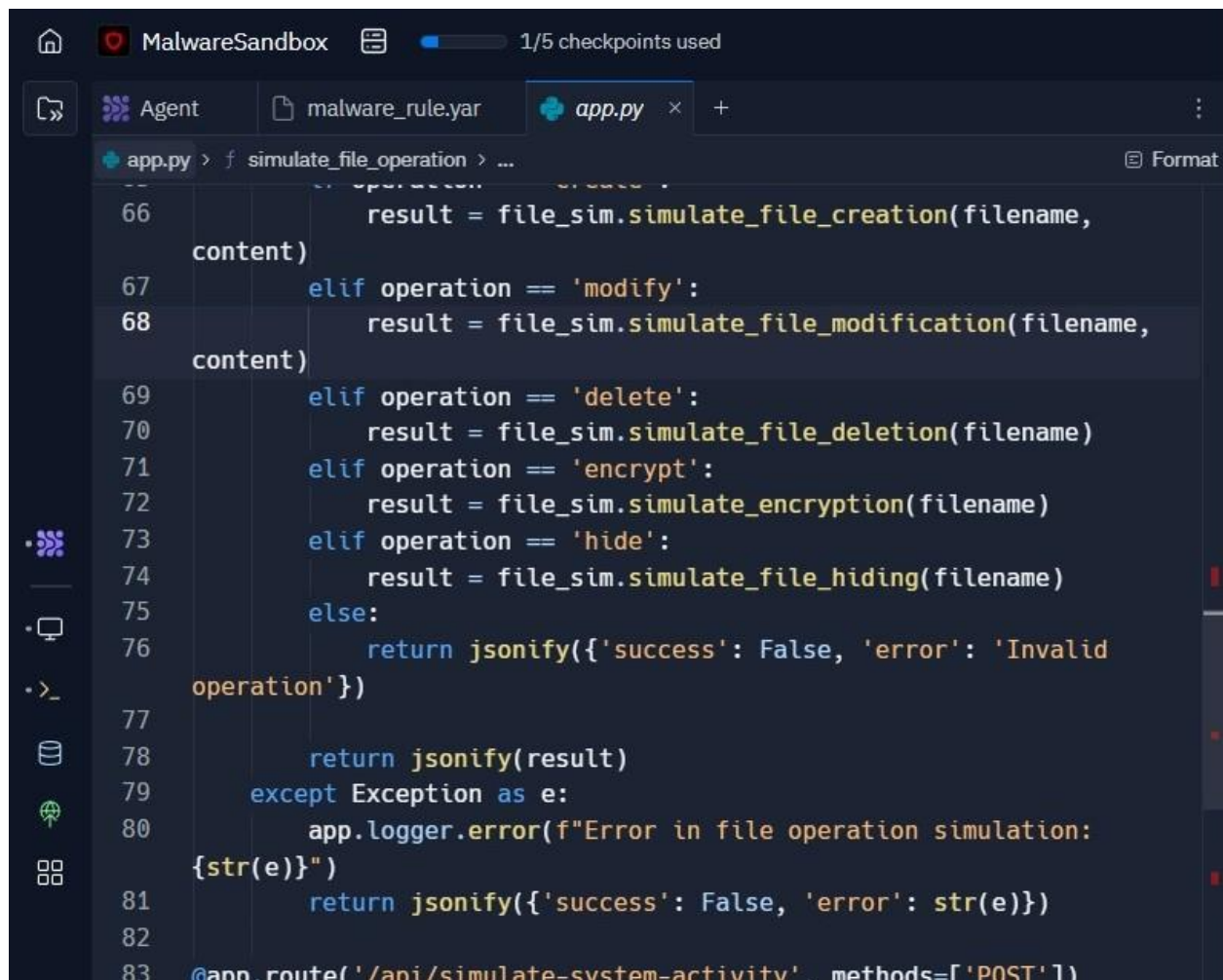
Step 4: Writing YARA Rule

The screenshot shows the MalwareSandbox application interface. At the top, there's a status bar with a home icon, the text 'MalwareSandbox', a list icon, a progress indicator '1/5 checkpoints used', and a tab bar. The tab bar contains three tabs: 'Agent', 'malware_rule.yar', and 'file_simulator.py' (which is active). Below the tabs, a breadcrumb trail shows the file path: 'simulator > file_simulator.py > FileSimulator > simulate_file_creation > ...'. A 'Format' button is visible on the right. The main area displays a Python script with line numbers 34 to 57. The script defines a method 'simulate_file_creation' that writes educational simulation content to a file. The content includes a timestamp, purpose, and safety statement, followed by a placeholder for the actual content. The script also shows the start of a dictionary definition for 'operation'.

```
purposes.\n"
34         )
35         f.write("All operations are completely safe and contained.\n")
36
37     def simulate_file_creation(self, filename: str,
38                               content: str) -> Dict[str, Any]:
39         """Simulate creating a file with educational content."""
40         try:
41             filepath = os.path.join(self.sandbox_dir, filename)
42
43             # Add educational metadata to content
44             enhanced_content = f"""# Educational Simulation File
45 # Created: {datetime.now().isoformat()}
46 # Purpose: Cybersecurity learning demonstration
47 # Safety: This is completely safe educational content
48
49 {content}
50
51 # End of educational content
52 """
53
54         with open(filepath, 'w') as f:
55             f.write(enhanced_content)
56
57         operation = {
```

- Created malware_rule.yar containing a custom rule:
- Rule Name: EducationalMalwareSim
- Author: Harsh
- Checks for:
 - “Educational Simulation File”
 - “ENCRYPTED_CONTENT”
- Fake command and control URL: “http://fake-c2-server.com”

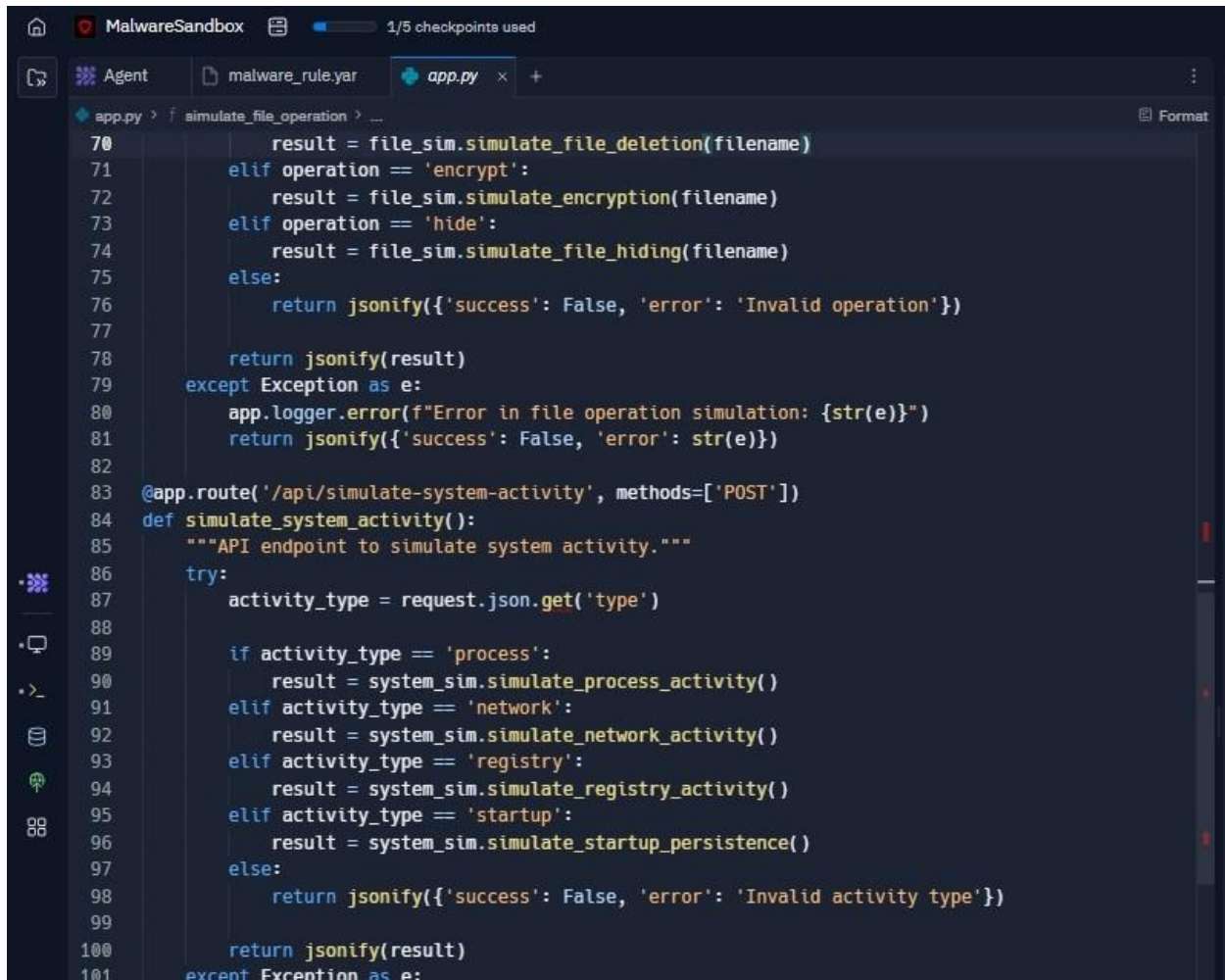
Step 5: Simulating File Creation



```
MalwareSandbox 1/5 checkpoints used
Agent malware_rule.yar app.py
app.py > f simulate_file_operation > ... Format
66         result = file_sim.simulate_file_creation(filename,
67         content)
68     elif operation == 'modify':
69         result = file_sim.simulate_file_modification(filename,
70         content)
71     elif operation == 'delete':
72         result = file_sim.simulate_file_deletion(filename)
73     elif operation == 'encrypt':
74         result = file_sim.simulate_encryption(filename)
75     elif operation == 'hide':
76         result = file_sim.simulate_file_hiding(filename)
77     else:
78         return jsonify({'success': False, 'error': 'Invalid
79         operation'})
80
81     return jsonify(result)
82 except Exception as e:
83     app.logger.error(f"Error in file operation simulation:
84     {str(e)}")
85     return jsonify({'success': False, 'error': str(e)})
86
87 @app.route('/api/simulate-system-activity', methods=['POST'])
```

- Function: simulate_file_creation()
- Generates a file in the sandbox with metadata.
- Logs the operation for visibility and future detection.

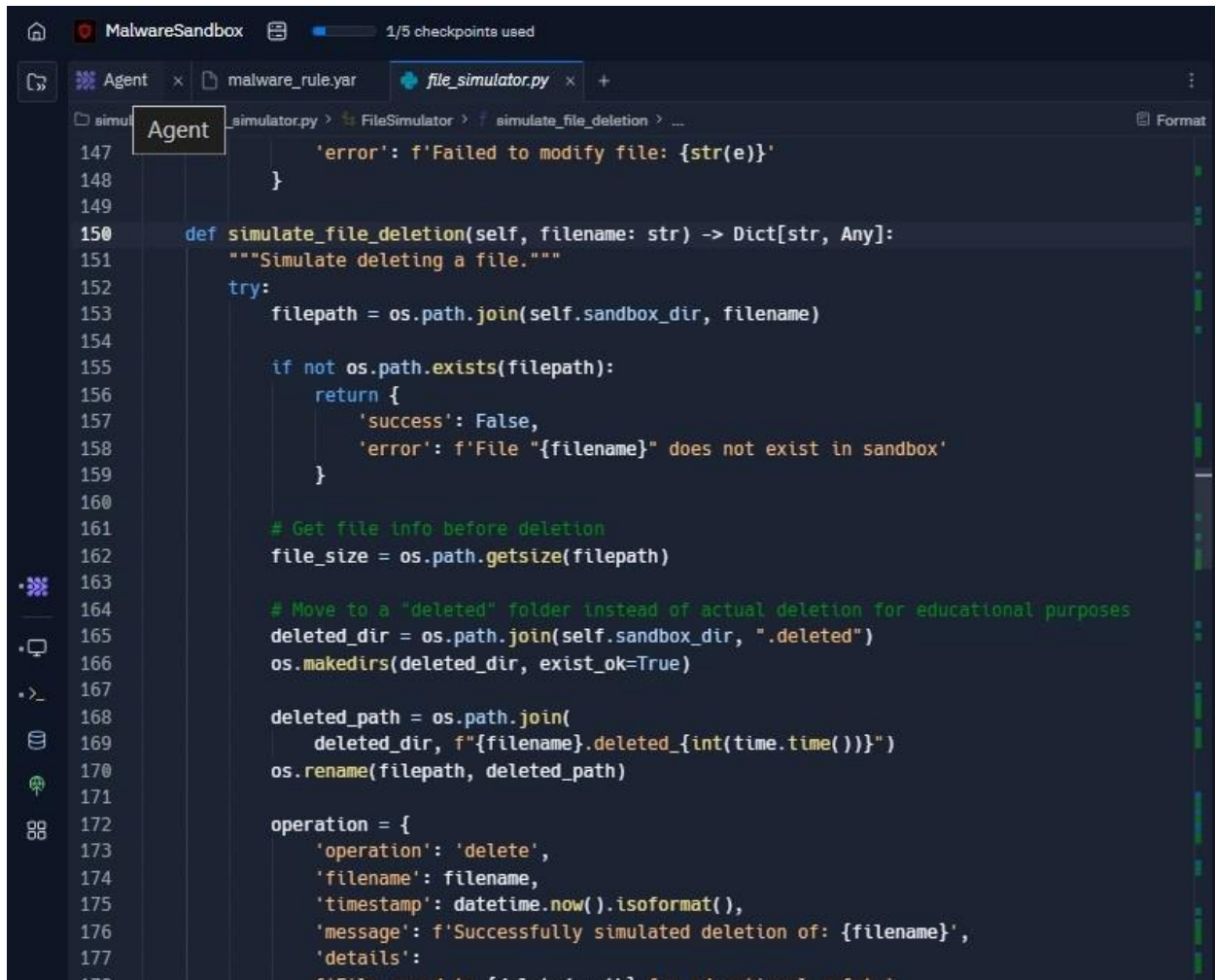
Step 6: Simulating File Modification, Encryption, and Hiding



```
MalwareSandbox 1/5 checkpoints used
Agent malware_rule.yar app.py x +
app.py > simulate_file_operation > ... Format
70     result = file_sim.simulate_file_deletion(filename)
71 elif operation == 'encrypt':
72     result = file_sim.simulate_encryption(filename)
73 elif operation == 'hide':
74     result = file_sim.simulate_file_hiding(filename)
75 else:
76     return jsonify({'success': False, 'error': 'Invalid operation'})
77
78     return jsonify(result)
79 except Exception as e:
80     app.logger.error(f"Error in file operation simulation: {str(e)}")
81     return jsonify({'success': False, 'error': str(e)})
82
83 @app.route('/api/simulate-system-activity', methods=['POST'])
84 def simulate_system_activity():
85     """API endpoint to simulate system activity."""
86     try:
87         activity_type = request.json.get('type')
88
89         if activity_type == 'process':
90             result = system_sim.simulate_process_activity()
91         elif activity_type == 'network':
92             result = system_sim.simulate_network_activity()
93         elif activity_type == 'registry':
94             result = system_sim.simulate_registry_activity()
95         elif activity_type == 'startup':
96             result = system_sim.simulate_startup_persistence()
97         else:
98             return jsonify({'success': False, 'error': 'Invalid activity type'})
99
100     return jsonify(result)
101 except Exception as e:
```

- app.py uses conditionals to invoke the following functions:
- simulate_file_modification()
- simulate_file_encryption()
- simulate_file_hiding()
- Each method logs simulated behavior in a safe manner.

Step 7: Simulating System Activities

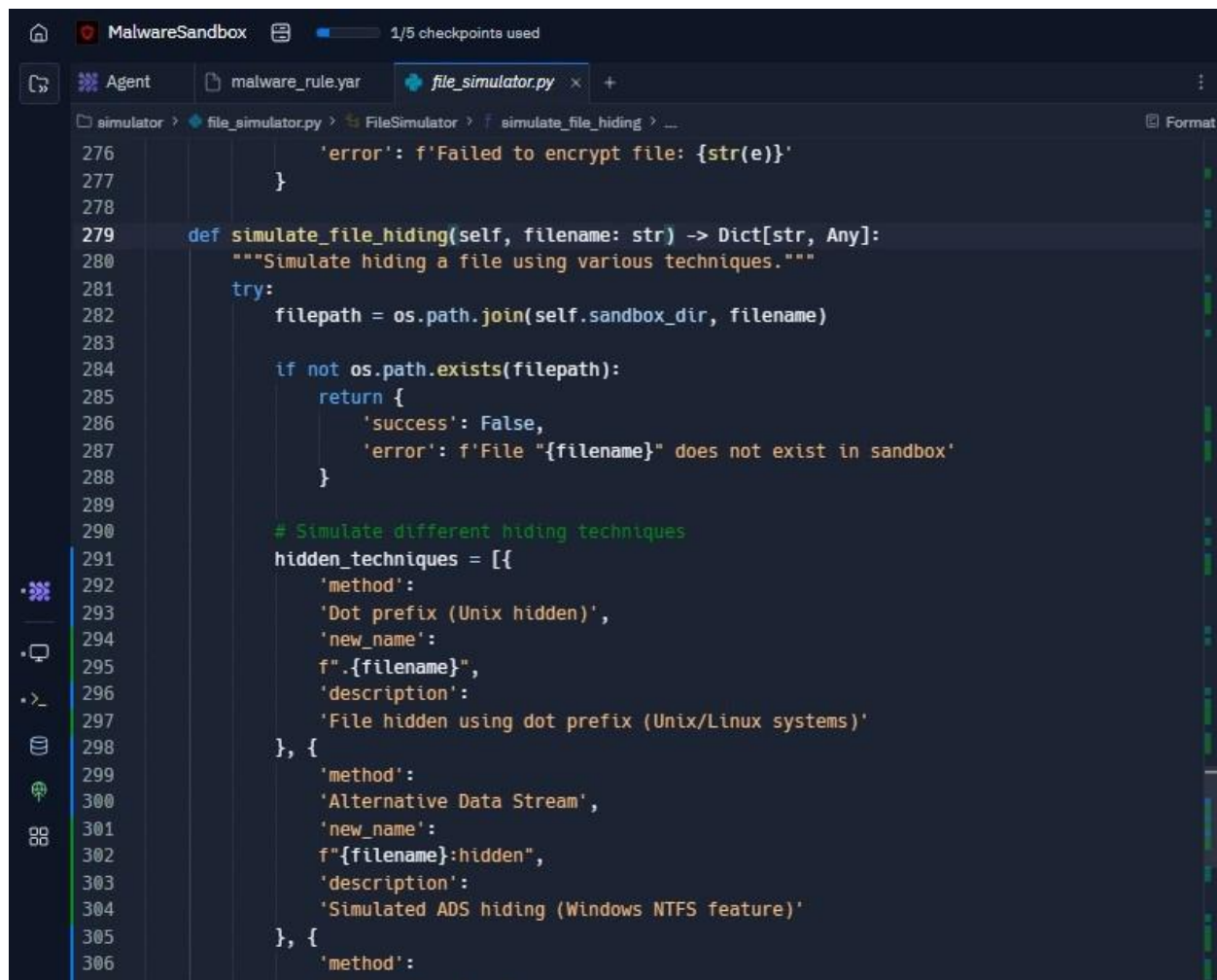


```
147         'error': f'Failed to modify file: {str(e)}'
148     }
149
150     def simulate_file_deletion(self, filename: str) -> Dict[str, Any]:
151         """Simulate deleting a file."""
152         try:
153             filepath = os.path.join(self.sandbox_dir, filename)
154
155             if not os.path.exists(filepath):
156                 return {
157                     'success': False,
158                     'error': f'File "{filename}" does not exist in sandbox'
159                 }
160
161             # Get file info before deletion
162             file_size = os.path.getsize(filepath)
163
164             # Move to a "deleted" folder instead of actual deletion for educational purposes
165             deleted_dir = os.path.join(self.sandbox_dir, ".deleted")
166             os.makedirs(deleted_dir, exist_ok=True)
167
168             deleted_path = os.path.join(
169                 deleted_dir, f"{filename}.deleted_{int(time.time())}")
170             os.rename(filepath, deleted_path)
171
172             operation = {
173                 'operation': 'delete',
174                 'filename': filename,
175                 'timestamp': datetime.now().isoformat(),
176                 'message': f'Successfully simulated deletion of: {filename}',
177                 'details':
178                     f'File moved to {deleted_path} for educational purposes'
```

- API routes simulate the following behaviors:

- Process creation
- Network communication
- Registry access
- Startup persistence simulation

Step 8: Simulating File Deletion

The image shows a Repl environment titled 'MalwareSandbox' with a status bar indicating '1/5 checkpoints used'. The file explorer on the left shows a project structure with 'Agent', 'malware_rule.yar', and 'file_simulator.py'. The main editor displays the code for 'file_simulator.py', specifically the 'simulate_file_hiding' function. The code includes error handling for file encryption failures, a check for file existence in the sandbox, and a list of simulated hiding techniques such as 'Dot prefix (Unix hidden)' and 'Alternative Data Stream' (ADS).

```
276         'error': f'Failed to encrypt file: {str(e)}'
277     }
278
279     def simulate_file_hiding(self, filename: str) -> Dict[str, Any]:
280         """Simulate hiding a file using various techniques."""
281         try:
282             filepath = os.path.join(self.sandbox_dir, filename)
283
284             if not os.path.exists(filepath):
285                 return {
286                     'success': False,
287                     'error': f'File "{filename}" does not exist in sandbox'
288                 }
289
290             # Simulate different hiding techniques
291             hidden_techniques = [{
292                 'method':
293                 'Dot prefix (Unix hidden)',
294                 'new_name':
295                 f".{filename}",
296                 'description':
297                 'File hidden using dot prefix (Unix/Linux systems)'
298             }, {
299                 'method':
300                 'Alternative Data Stream',
301                 'new_name':
302                 f"{filename}:hidden",
303                 'description':
304                 'Simulated ADS hiding (Windows NTFS feature)'
305             }, {
306                 'method':
```

- Function: `simulate_file_deletion()`
- Instead of permanently deleting the file, it is moved to a `.deleted` folder.
- Ensures safety and traceability for educational review.

Conclusion

This project effectively demonstrates simulated malware behavior such as file encryption, modification, deletion, and system activity. All actions are performed safely within a controlled Repl environment. The custom YARA rule accurately detects these behaviors based on predefined string patterns.

This setup provides an ideal learning experience for cybersecurity students and researchers.

Deliverables

- Fully functioning Flask app on Replit
- Custom YARA rule file: `malware_rule.yar`
- Screenshots documenting the full development process

Prepared By: Harsh

Date: June 1, 2025